```
import "dotenv/config";
import express from "express";
import cors from "cors";
import { Low } from "lowdb";
import { JSONFile } from "lowdb/node";
import Stripe from "stripe";

const PORT = process.env.PORT || 4000;
const stripe = process.env.STRIPE_SECRET_KEY
  ? new Stripe(process.env.STRIPE_SECRET_KEY)
  : null;

/* ---------------------------------- database ----------------------------------
// A plain JSON file on disk — no native compilation required, unlike better-sqli
// which is what was breaking the Render build. NOTE: on Render's free tier the
// filesystem is ephemeral — this resets on every redeploy/restart. Fine for
// testing; move to a real managed Postgres (Render, Supabase, Neon) before you
// have real customers depending on this data.

const adapter = new JSONFile("quiktow-db.json");
const db = new Low(adapter, { jobs: [] });
await db.read();
db.data ||= { jobs: [] };

/* ---------------------------------- app ---------------------------------- */

const app = express();
app.use(cors()); // tighten this to your real frontend origin before launch
app.use((req, res, next) => {
  // Stripe webhook needs the raw body for signature verification, so skip JSON
  // parsing for that one route.
  if (req.originalUrl === "/api/stripe/webhook") return next();
  express.json()(req, res, next);
});

function makeJobId() {
  return "QT" + Math.floor(1000 + Math.random() * 9000);
}

app.get("/api/health", (req, res) => {
  res.json({ ok: true, stripeConfigured: Boolean(stripe) });
});

// List all jobs, most recent first.
```

```
app.get("/api/jobs", async (req, res) => {
  await db.read();
  const jobs = [...db.data.jobs].sort((a, b) => b.created_at - a.created_at);
  res.json(jobs);
});

// Create a new job request (customer app).
app.post("/api/jobs", async (req, res) => {
  const { service, priceValue, pickup, customer } = req.body;
  if (!service || typeof priceValue !== "number") {
    return res.status(400).json({ error: "service and numeric priceValue are requ
  }
  const job = {
    id: makeJobId(),
    service,
    price_value: priceValue,
    status: "requested",
    customer: customer || "Guest Customer",
    driver: null,
    pickup: pickup || "",
    stripe_payment_intent: null,
    created_at: Date.now(),
  };
  await db.read();
  db.data.jobs.push(job);
  await db.write();
  res.status(201).json(job);
});

// Update job status / assign a driver (driver app + dispatch).
app.patch("/api/jobs/:id", async (req, res) => {
  const { status, driver } = req.body;
  await db.read();
  const job = db.data.jobs.find((j) => j.id === req.params.id);
  if (!job) return res.status(404).json({ error: "job not found" });

  if (status !== undefined) job.status = status;
  if (driver !== undefined) job.driver = driver;
  await db.write();

  res.json(job);
});

// Create a real Stripe PaymentIntent for a job (test mode until you swap keys).
app.post("/api/jobs/:id/pay", async (req, res) => {
  if (!stripe) return res.status(500).json({ error: "Stripe is not configured on
  await db.read();
```

```
  const job = db.data.jobs.find((j) => j.id === req.params.id);
  if (!job) return res.status(404).json({ error: "job not found" });

  try {
    const intent = await stripe.paymentIntents.create({
      amount: Math.round(job.price_value * 100), // cents
      currency: "usd",
      metadata: { jobId: job.id },
      automatic_payment_methods: { enabled: true },
    });
    job.stripe_payment_intent = intent.id;
    await db.write();
    res.json({ clientSecret: intent.client_secret });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// Stripe webhook — this is the source of truth for "did the payment actually wor
// Never mark a job paid just because the frontend says so; wait for this event.
app.post("/api/stripe/webhook", express.raw({ type: "application/json" }), async
  if (!stripe || !process.env.STRIPE_WEBHOOK_SECRET) {
    return res.status(500).send("Webhook not configured");
  }
  let event;
  try {
    event = stripe.webhooks.constructEvent(
      req.body,
      req.headers["stripe-signature"],
      process.env.STRIPE_WEBHOOK_SECRET
    );
  } catch (err) {
    return res.status(400).send(`Webhook signature verification failed: ${err.mes
  }

  if (event.type === "payment_intent.succeeded") {
    const intent = event.data.object;
    const jobId = intent.metadata?.jobId;
    if (jobId) {
      await db.read();
      const job = db.data.jobs.find((j) => j.id === jobId);
      if (job) {
        job.status = "paid";
        await db.write();
      }
    }
  }
```

```
  res.json({ received: true });
});

app.listen(PORT, () => {
  console.log(`QuikTow API listening on port ${PORT}`);
  if (!stripe) console.warn("STRIPE_SECRET_KEY not set — payment endpoints will f
});
```